

AI记账-架构设计文档

爱记账 软件架构设计文档

1. 引言

1.1 文档目的

本文档旨在描述“爱记账”软件的整体架构设计，包括系统的主要组成部分、它们之间的关系、关键设计决策以及重要的非功能性属性。本文档将作为开发团队进行详细设计和实现的基础。

1.2 系统概述

“爱记账”是一款个人记账应用，旨在通过创新的 AI 问答功能，简化用户的记账流程。用户可以通过自然语言与应用交互进行记账、查询和生成消费报告。同时，应用也提供传统的列表方式查看和管理记账明细，并支持多种维度的数据可视化分析。

1.3 目标读者

- 开发团队
- 测试团队
- 评审人员

1.4 术语表

术语	定义
记账条目	一条收入或支出的记录
AI 问答	用户通过自然语言与应用进行交互的方式
NLP	自然语言处理
数据可视化	将数据以图表形式展示
大类标签	对记账条目的分类（如：餐饮、交通、工资等）
敏感内容识别	识别并提示违法乱纪的记账内容

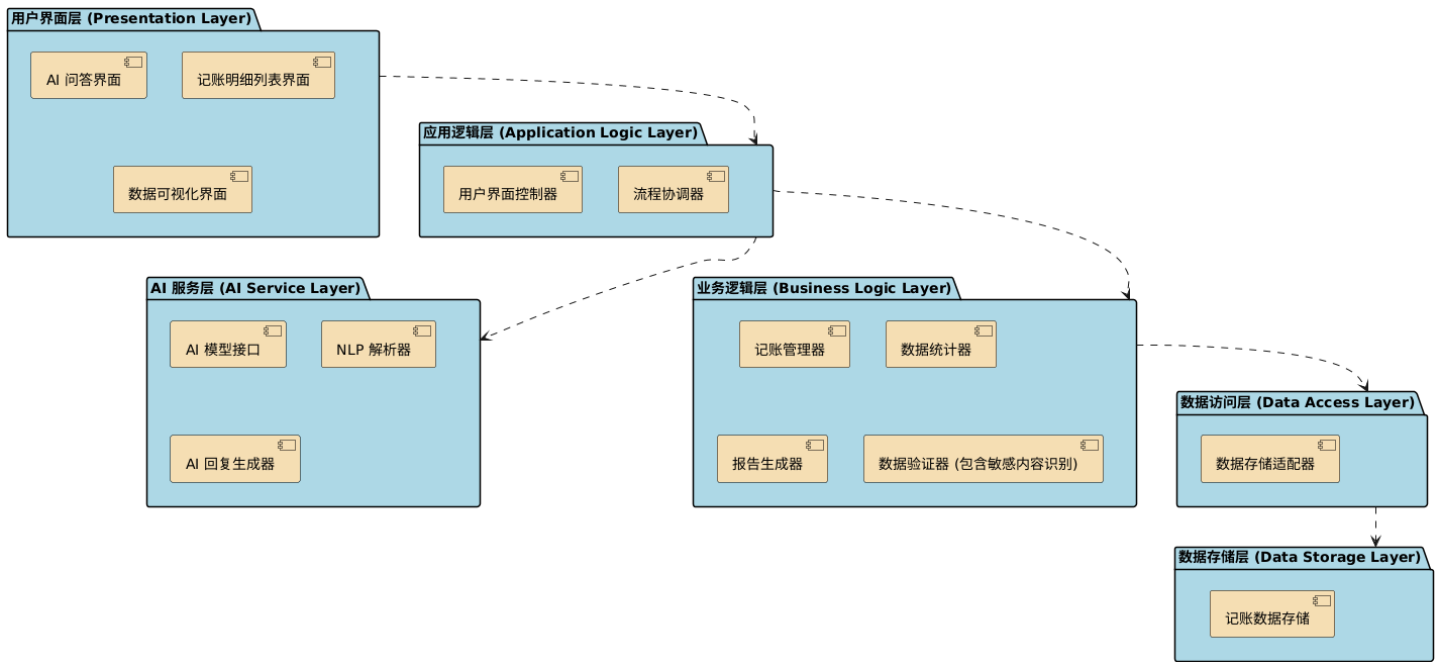
2. 架构视图

我们将采用**逻辑视图**、**开发视图**和**运行时视图**来描述“爱记账”的架构。

2.1 逻辑视图

逻辑视图描述系统的抽象结构，关注系统的主要功能块以及它们之间的依赖关系。

图示： 逻辑视图分层图



- 用户界面层 (Presentation Layer):
 - AI 问答界面
 - 记账明细列表界面
 - 数据可视化界面
- 应用逻辑层 (Application Logic Layer):
 - 用户界面控制器
 - 流程协调器
- 业务逻辑层 (Business Logic Layer):
 - 记账管理器
 - 数据统计器
 - 报告生成器
 - 数据验证器 (包含敏感内容识别)
- AI 服务层 (AI Service Layer):
 - AI 模型接口
 - NLP 解析器
 - AI 回复生成器
- 数据访问层 (Data Access Layer):
 - 数据存储适配器

- 数据存储层 (Data Storage Layer):
 - 记账数据存储

层间依赖关系描述：

- 用户界面层依赖于应用逻辑层。
- 应用逻辑层依赖于业务逻辑层和 AI 服务层。
- 业务逻辑层依赖于数据访问层。
- AI 服务层依赖于外部 AI API 。
- 数据访问层依赖于数据存储层。

2.2 开发视图

开发视图描述系统的代码组织结构，方便开发人员理解代码库。

图示： 开发视图包图/模块图

[这里等着进行开发后补充]

可能的模块划分：

- `com.aizhang.ui` (用户界面模块)
- `com.aizhang.applogic` (应用逻辑模块)
- `com.aizhang.businesslogic` (业务逻辑模块)
- `com.aizhang.aiservice` (AI 服务模块)
- `com.aizhang.dataaccess` (数据访问模块)
- `com.aizhang.datastorage` (数据存储模块)
- `com.aizhang.common` (通用工具类和模型)

模块间依赖关系描述：

- UI 模块依赖于 App Logic 模块。
- App Logic 模块依赖于 Business Logic 模块和 AI Service 模块。
- Business Logic 模块依赖于 Data Access 模块。
- Data Access 模块依赖于 Data Storage 模块。
- 所有模块可能都依赖于 Common 模块。

2.3 运行时视图【等待化用丁桑的时序图】

运行时视图描述系统在运行时的进程、线程和对象交互。这里展示最为复杂的AI记账时序图。

图示： 通过AI问答记账的时序图

[这里应该有一张时序图，展示用户通过 AI 问答进行记账的详细交互流程，包括用户、UI、App Logic、AI Service、Business Logic、Data Access 和 Data Storage 之间的消息传递和调用顺序，类

似于前面示例流程的图示化。]

文字描述示例（对应时序图）：

1. 用户在 UI 输入记账文本。
2. UI 将文本发送给 App Logic。
3. App Logic 调用 AI Service 的 NLP 解析方法。
4. AI Service 调用 AI 模型进行文本解析。
5. AI 模型返回解析结果（记账意图、金额等）。
6. AI Service 将解析结果返回给 App Logic。
7. App Logic 调用 Business Logic 的记账处理方法。
8. Business Logic 进行数据验证（包括敏感内容识别）。
9. 如果验证通过，Business Logic 调用 Data Access 的保存方法。
10. Data Access 与 Data Storage 交互，保存记账数据。
11. Data Access 返回保存结果给 Business Logic。
12. Business Logic 返回处理结果给 App Logic。
13. App Logic 调用 AI Service 生成回复文本。
14. AI Service 返回回复文本给 App Logic。
15. App Logic 更新 UI 并显示 AI 回复。

3. 组件设计

本节详细描述系统中的主要组件及其职责。

3.1 用户界面组件

- **AI 问答界面组件：** 负责 AI 聊天和记账问答的 UI 展示和用户输入处理，以及调用消费报告的生成。
- **记账明细列表界面组件：** 负责记账条目的列表展示、筛选和排序，以及对于每一个条目触发修改和删除操作。
- **记账条目编辑/删除组件：** 提供修改和彻底删除记账条目的界面和逻辑。
- **数据可视化界面组件：** 负责根据用户数据的筛选条件，进行不同类型图表的展示。

3.2 应用逻辑组件

- **用户界面控制器：** 负责处理用户界面层的事件，协调业务逻辑和 AI 服务。
- **流程协调器：** 管理跨多个组件的复杂操作流程，例如通过 AI 问答完成记账的整个流程。

3.3 业务逻辑组件

- **记账管理器：** 管理记账条目的生命周期（创建、读取、更新、删除），处理记账相关的业务规则。

- **数据统计器：** 根据用户的筛选条件，进行记账数据的汇总和统计计算（总收入、总支出、净收入、按类别统计等）。
- **报告生成器：** 根据统计结果生成消费报告（周、月、年）。
- **数据验证器：** 对用户输入的记账数据进行验证，包括金额格式、日期有效性以及敏感内容的识别。

3.4 AI 服务组件

- **AI 模型接口：** 封装与实际 AI 模型（本地或云端）的交互逻辑。
- **NLP 解析器：** 使用自然语言处理技术，从用户文本中提取记账相关的关键信息（金额、类型、日期、描述）。
- **AI 回复生成器：** 根据处理结果和用户请求，生成自然语言的回复文本。

3.5 数据访问组件

- **数据存储适配器：** 提供统一的数据访问接口，将业务逻辑与具体的数据存储实现解耦。根据底层数据存储方式，实现不同的适配器。

3.6 数据存储组件

- **记账数据存储：** 负责记账数据的持久化。使用本地文件的方式（如 JSON 或 CSV）。

4. 关键设计决策

- **AI 集成方式：**
 - **决策：** 调用第三方 AI API
 - **理由：**
 - **实现难度低：** 调用成熟的第三方 AI API 可以极大地降低开发难度和时间成本，无需从头构建和训练 AI 模型。
 - **性能和效果：** 第三方大型 AI API 通常经过大量数据训练和优化，在自然语言理解和处理方面具有更好的性能和效果，能够更准确地识别记账信息和提供高质量的聊天回复。
 - **无需维护模型：** 第三方服务提供商负责模型的维护、更新和基础设施管理，开发者无需投入精力。
 - **快速迭代：** 可以快速集成最新的 AI 能力，无需等待本地模型训练和部署。
 - **成本可控：** 对于个人或小型应用，按量付费的 API 调用成本可能比自建和维护 AI 基础设施更具成本效益。
- **数据存储方案：**
 - **决策：** 本地文件（JSON）
 - **理由：**
 - **简单易实现：** JSON 格式易于理解和处理，使用文件存储是最简单的数据持久化方式，适合快速开发和简单的应用场景。

- **无需额外依赖：** 本地文件存储不需要额外的数据库软件或配置，部署和使用非常方便。
- **适用于小型数据量：** 对于个人记账这样数据量相对较小的应用，文件存储的性能通常可以满足需求。
- **易于查看和编辑：** JSON 文件是文本格式，用户或开发者可以直接打开和查看数据，方便调试和数据管理。
- **可移植性：** JSON 文件易于在不同平台和设备之间迁移。

- **敏感内容识别策略：**

- **决策：** 采用 AI 大模型调用来识别敏感内容。
- **理由：**
 - **更高的准确率：** 相比于简单的关键词匹配或正则表达式，大型 AI 模型在理解文本语境和识别隐含的敏感信息方面具有更强的能力，可以有效降低误报和漏报率。
 - **更全面的识别范围：** AI 大模型能够识别更广泛的敏感内容类型，而不仅仅局限于预设的关键词列表。
 - **无需手动维护规则：** 无需手动维护和更新复杂的敏感词库和规则，降低维护成本。
 - **与 AI 问答功能集成：** 可以复用 AI 问答所使用的 AI 模型，降低额外的技术栈引入。
 - **应对变种和新出现的敏感词：** AI 模型具有一定的泛化能力，能够识别敏感词的变种或新出现的敏感表达。

- **数据可视化库选择【待定】：**

- **决策：** 使用 [选择一个图表库，例如：Chart.js / ECharts / 其他]。
- **理由：** [说明选择该库的原因，考虑功能丰富度、易用性、社区支持和许可证等因素]。

- **错误处理和日志记录【待定】：**

- **决策：** 如何处理各类错误（如AI服务调用失败、数据存储异常），以及如何进行日志记录以便调试和问题排查。

5. 数据模型

- **记账条目 (LedgerEntry)【待定，仅作参考】**

```
class LedgerEntry {
    int id; // 唯一标识
    DateTime time; // 年月日时分
    string categoryTag; // 大类标签
    string specificName; // 具体名字
    decimal amount; // 金额 (+ 收入, - 支出)
}
```

6. 接口定义【待定 仅作参考】

本节定义主要组件之间的接口，以便于模块的解耦和并行开发。

- **ILedgerManager** (业务逻辑层提供给应用逻辑层)
 - `AddEntry(LedgerEntry entry)` : 添加记账条目。
 - `UpdateEntry(LedgerEntry entry)` : 更新记账条目。
 - `DeleteEntry(int entryId)` : 删除记账条目。
 - `GetEntries(DateTime startDate, DateTime endDate, List<string> categories, bool includeIncome, bool includeExpense)` : 根据条件获取记账条目列表。
 - `CalculateTotalIncome(...)` : 计算总收入。
 - `CalculateTotalExpense(...)` : 计算总支出。
 - `CalculateBalance(...)` : 计算余额。
 - `GenerateReport(...)` : 生成消费报告。
- **IAIService** (AI 服务层提供给应用逻辑层)
 - `ProcessNaturalLanguage(string text)` : 处理自然语言文本，返回解析结果（包含记账信息或聊天回复）。
 - `IdentifySensitiveContent(string text)` : 识别文本中的敏感内容。
- **IDataStore** (数据访问层提供给业务逻辑层)
 - `SaveEntries(List<LedgerEntry> entries)` : 保存记账条目列表。
 - `LoadEntries()` : 加载所有记账条目。
 - `AddEntry(LedgerEntry entry)` : 添加单个记账条目。
 - `UpdateEntry(LedgerEntry entry)` : 更新单个记账条目。
 - `DeleteEntry(int entryId)` : 删除单个记账条目。

7. 质量属性考虑

- **可用性**：保证软件在用户设备上的稳定运行，提供友好的用户体验。
- **性能**：确保记账、查询和可视化操作的快速响应，尤其是在数据量增长时。
- **可维护性**：通过分层和模块化设计，降低代码的复杂性，提高修改和扩展的效率。
- **安全性**：保护用户记账数据的隐私和安全。对敏感内容进行识别和提示，避免记录违法乱纪信息。
- **可扩展性**：架构应支持未来可能增加的记账类型、报告维度或 AI 功能。
- **准确性**：确保记账数据的记录、计算和统计结果的准确性。